ТОО «Колледж Хекслет»

КУРСАВАЯ РАБОТА

на тему: «Разработка веб-приложения "GoalTracker" для отслеживания личных целей»

по модулю: ПМ2 «Прикладное программирование»
Специальность: 06130100 – Программное обеспечение (по видам)
Квалификация: 4S06130105 – Техник информационных систем

Выполнил: студент 2 курса
13 ТИС группы
Степанов Тим Александрович

Допущен(-а) к защите «___» _____ 20___ г.

Преподаватель: _____
(подпись)

Защитил(-а) с оценкой:

_____
(буква) (циф.экв.) (балл)

Алматы 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 2
1. **ТЕОРЕТИЧЕСКАЯ ЧАСТЬ** 3
Анализ предметной области и функциональных требований 5
Проектирование интерфейса форм и навигации 8
Обоснование выбора хранилища данных и библиотек 11
2. **ПРАКТИЧЕСКАЯ ЧАСТЬ** 14
Реализация программной логики на JavaScript 14
Интеграция хранилища данных и обработка событий 18
Тестирование и отладка приложения 21
ЗАКЛЮЧЕНИЕ 23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 24
ПРИЛОЖЕНИЯ 25

ВВЕДЕНИЕ

В условиях современного информационного общества эффективное управление личными целями приобретает особую значимость. Умение формулировать цели, планировать их достижение и своевременно оценивать прогресс — ключевые компетенции успешного специалиста в сфере информационных технологий.

Учебная практика по профессиональному модулю ПМ2 «Прикладное программирование» направлена на практическое применение знаний в области веб-разработки. В ходе практики разработано веб-приложение GoalTracker — система постановки и отслеживания личных целей.

Актуальность темы обусловлена высоким спросом на инструменты личной продуктивности. Приложения для управления целями и задачами входят в топ наиболее загружаемых категорий в магазинах приложений. Разработка подобной системы охватывает весь спектр клиентских веб-технологий: HTML5, CSS3 и JavaScript ES6+.

Цель практики:

Разработать веб-приложение «GoalTracker» — систему постановки и отслеживания личных целей с возможностью категоризации, установки приоритетов и дедлайнов, отображения прогресса и просмотра статистики.

Задачи практики:

– изучить клиентские веб-технологии HTML5, CSS3, JavaScript ES6+;
– спроектировать структуру данных для хранения целей в localStorage;
– разработать модульную архитектуру клиентского приложения;
– реализовать операции CRUD для работы с целями;
– создать адаптивный интерфейс с поиском и фильтрацией;
– реализовать модуль статистики;
– обеспечить валидацию данных и корректную обработку ошибок.

Ожидаемые результаты:

Функциональное веб-приложение, работающее в браузере без серверной части, с сохранением данных между сессиями и полным набором инструментов для управления личными целями. Приобретение практических навыков в области HTML5, CSS3, JavaScript ES6+ и Web Storage API.

Методы работы:

– изучение официальной документации MDN Web Docs;
– применение принципов модульного программирования;– итеративная разработка с тестированием каждого компонента; – использование Git и GitHub для контроля версий.

1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Анализ предметной области и функциональных требований

Предметная область проекта — **управление личными целями**. Пользователь формулирует цели, определяет их категорию, приоритет и срок выполнения, фиксирует прогресс и отмечает достижение. Система хранит совокупность целей и предоставляет инструменты фильтрации, поиска и аналитики. (таблица 1)

Таблица 1 – Сравнение аналогов

№	Система	Тип	Хранение	Особенности
1	*2*	*3*	*4*	*5*
1	Todoist	Облачная	Серверная БД	Многопользовательская
2	Habitica	Облачная	Серверная БД	Геймификация
3	GoalTracker	Клиентская	localStorage	Не требует сервера
Примечание: Таблица составлена на основе анализа открытых источников. GoalTracker выбран как клиентское решение, не требующее серверной инфраструктуры				

Функциональные требования

– ФТ-1: Просмотр списка всех целей на главной странице.
– ФТ-2: Создание цели через форму (название, категория, дедлайн,приоритет, прогресс, описание).
– ФТ-3: Удаление цели с подтверждением.
– ФТ-4: Переключение статуса цели (активна / выполнена).
– ФТ-5: Поиск целей по названию в режиме реального времени.
– ФТ-6: Фильтрация по категории (5 вариантов).
– ФТ-7: Фильтрация по статусу (все / активные / выполненные).
– ФТ-8: Просмотр статистики: итоги, процент выполнения, топ-категория.
– ФТ-9: Автоинициализация тестовыми данными при первом запуске.

Нефункциональные требования

– НФТ-1: Время загрузки страницы — не более 2 секунд.
– НФТ-2: Отклик на действие пользователя — не более 100 мс.

– НФТ-3: Поддержка браузеров: Chrome 90+, Firefox 88+, Edge 90+.

– НФТ-4: Адаптивный дизайн для экранов от 320px до 1920px.

– НФТ-5: Работа в offline-режиме (кроме Google Fonts).

– НФТ-6: Модульная организация кода.

Описание объекта Goal-Каждая цель представлена объектом Goal со следующим набором полей, описанных в Таблице 2.

Таблица 2 -поля объекта Goal

№	**Поле**	**Тип**	**Обязательное**	**Описание**
1	*1*	*2*	*3*	*4*
1	id	string	Авто	Уникальный идентификатор
2	title	string	Да	Название цели
3	category	string	Да	Категория (5 значений)
4	deadline	string	Да	Срок (YYYY-MM-DD)
5	priority	string	Да	Low / Medium / High
6	progress	number	Да	Прогресс 0–100%
7	description	string	Нет	Описание
8	completed	boolean	Авто	Статус выполнения
9	createdAt	string	Авто	Дата создания
Примечание: Поля id, completed и createdAt формируются автоматически при создании цели. Поле description является необязательным.				

Проектирование интерфейса форм и навигации

Приложение GoalTracker состоит из четырёх HTML-страниц, связанных единой навигационной панелью. Пользователь перемещается между страницами через меню, активная страница визуально выделена. Навигация реализована через обычные гиперссылки без использования роутера. Подробный состав страниц приведён в Таблице 3.

Таблица 3 - Страницы приложения GoalTracker

№	Страница	Файл	Назначение	Загружаемые модули
1	*1*	*2*	*3*	*4*
1	Главная	index.html	Список и управление целями	storage.js, utils.js, main.js
2	Добавить	form.html	Создание новой цели	storage.js, utils.js, form.js
3	О проекте	about.html	Информация о приложении	storage.js, utils.js
4	Статистика	stats.html	Аналитика по целям	storage.js, utils.js, stats.js
Примечание: Каждая страница подключает только необходимые JS-модули, что минимизирует нагрузку на браузер.				

Проектирование формы создания цели (form.html)

Форма создания цели — центральный UI-элемент приложения. Она содержит шесть полей ввода, кнопки «Сохранить» и «Отмена», встроенную валидацию на уровне HTML5-атрибутов (required, min, max, type) и дополнительную JS-валидацию. Поля формы описаны в Таблице 4.

Таблица 4- Поля формы создания цели

№	Поле	HTML-элемент	Атрибуты	Описание
1	*1*	*2*	*3*	*4*
1	Название	input[text]	required	Название цели
2	Категория	select	required	5 категорий
3	Дедлайн	input[date]	required	Дата YYYY-MM-DD
4	Приоритет	select	required	Low/Medium/High
5	Прогресс	input[number]	min=0 max=100	Процент 0–100
6	Описание	textarea	—	Необязательное
Примечание: Поле «Описание» является необязательным. Валидация остальных полей выполняется как на уровне HTML5-атрибутов, так и средствами JavaScript.				

Проектирование главной страницы (index.html)

Главная страница состоит из трёх функциональных зон:

– Зона фильтрации: текстовый поиск (#search), выбор категории (#categoryFilter), выбор статуса (#statusFilter). Все три фильтра работают совместно и применяются мгновенно при изменении;

– Зона карточек: CSS Grid-сетка с карточками целей (#goalsContainer). Каждая карточка содержит все атрибуты цели, прогресс-бар и кнопки действий;

– Кнопка добавления: плавающая кнопка перехода на форму создания цели.

Цветовая схема и типографика- Для обеспечения визуальной согласованности интерфейса была разработана единая цветовая схема, представленная в Таблице 5

Таблица 5- Цветовая схема интерфейса GoalTracker

№	Элемент	Цвет (HEX)	Назначение
1	***1***	***2***	***3***
1	Шапка / навигация	#1f3c88	Тёмно-синий (брендовый)
2	Основные кнопки	#3b82f6	Синий (действия)
3	Успех / добавление	#22c55e	Зелёный (подтверждение)
4	Удаление	#ef4444	Красный (деструктивное)
5	Фон страницы	#f4f7fb	Светло-голубой
6	Карточки	#ffffff	Белый (контент)
Примечание: Все цвета подобраны с учётом достаточного контраста для обеспечения читаемости (WCAG AA). Брендовый цвет #1f3c88 используется в шапке и навигации.			

Адаптивный дизайн

Реализован один брейкпоинт: @media (max-width: 768px). При ширине экрана менее 768px:

– навигационные ссылки переносятся и выравниваются вертикально;

– элементы фильтрации занимают 100% ширины и располагаются встолбец;

– сетка карточек переключается в одноколоночный вид;– кнопки формы выстраиваются вертикально.

Обоснование выбора хранилища данных и библиотек

Анализ вариантов хранения данных - При проектировании системы хранения данных были рассмотрены три варианта. Их сравнительный анализ приведён в Таблице 6.

Таблица 6- Варианты хранения данных

№	Технология	Преимущества	Недостатки
1	***1***	***2***	***3***
1	localStorage	Без сервера, простота	Только один браузер
2	IndexedDB	Структурированные данные	Сложный API
3	Сервер + MySQL	Многопользовательский	Требует бэкенд
Примечание: По результатам сравнения для учебного проекта выбран localStorage как наиболее простой и самодостаточный вариант, не требующий серверной инфраструктуры.			

Обоснование выбора localStorage

Для данного учебного проекта выбран **localStorage** как наиболее подходящий вариант по следующим причинам:

- Не требует серверной части — приложение полностью работает в браузере;
- Персистентность — данные сохраняются между закрытием вкладок и перезапуском браузера;
- Достаточный объём — до 5 МБ, что хватает для хранения сотен объектов Goal;
- Простой API — getItem() / setItem() с JSON-сериализацией;

– Синхронный доступ — данные доступны мгновенно без Promise/async.

Схема хранилища данных

В localStorage используется единственный ключ **"goals"**, значение которого — JSON-массив объектов Goal:

```
localStorage["goals"] = JSON.stringify([
{
    "id":          "1712345678abcdef",
    "title":       "■■■■■■■ JavaScript",
    "category":    "Study",
    "deadline":    "2026-06-01",
    "priority":    "High",
"progress":    45,
    "description": "■■■■■■■ ES6+",
    "completed":   false,
"createdAt":   "2026-01-15"
},   ... ]);
```

Описание категорий - Для удобства классификации целей в приложении предусмотрено пять категорий, описанных в Таблице 7.

Таблица 7 -Категории целей в приложении GoalTracker.

№	**Значение (ключ)**	**Название (UI)**	**Описание**
1	***1***	***2***	***3***
2	Study	Учёба	Образовательные и учебные цели
3	Sport	Спорт	Физическое развитие и тренировки
4	Health	Здоровье	Здоровый образ жизни
5	Finance	Финансы	Накопления и финансовые цели
6	Self-development	Саморазвитие	Личностный рост и навыки
Примечание: Значения ключей (value) используются в коде; названия UI отображаются пользователю в выпадающем списке формы и на карточках целей.			

Обоснование выбора библиотек

Проект намеренно разработан без внешних JavaScript-фреймворков.

Использован Vanilla JS (ES6+) — это позволяет:

– избежать избыточных зависимостей для учебного проекта;

– продемонстрировать чистые знания базового JavaScript;

– обеспечить быструю загрузку без бандлера и транспилятора.

Единственная внешняя зависимость — шрифт **Inter** из Google Fonts (подключается через <link> в HTML). Для генерации PDF-документации используется пакет markdown-pdf (npm).

2. ПРАКТИЧЕСКАЯ ЧАСТЬ

Реализация программной логики на JavaScript

Создание проекта и структуры файлов

Первым этапом было создание структуры проекта и репозитория на GitHub. Все JS-файлы размещены в папке js/, CSS — в css/. Каждый HTML-файл подключает только нужные модули.

```
project/ projec-
folder/
  index.html     # ■■■■■■■ ■■■■■■■■
form.html      # ■■■■■ ■■■■■■■■
■■■■  about.html     # ■ ■■■■■■■
stats.html     # ■■■■■■■■■■  css/
    style.css    # ■■■ ■■■■■
js/
    storage.js   # ■■■■ ■■■■■■
utils.js     # ■■■■■■■    main.js
# ■■■■■■■■ ■■■■■■    form.js
# ■■■■■■■■■ ■■■■■    stats.js
# ■■■■■■■■■■
  package.json   # npm (■■■ ■■■■■■■■■■■■■)
```

Модуль storage.js — слой данных

Все операции с localStorage инкапсулированы в storage.js. Остальные модули не обращаются к localStorage напрямую — только через функции этого модуля.

```
const STORAGE_KEY = 'goals';

function getGoals() {
  const data =
localStorage.getItem(STORAGE_KEY);   return data
? JSON.parse(data) : []; }

function saveGoals(goals) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(goals));
}

function initializeGoals() {
  if (localStorage.getItem(STORAGE_KEY)) return;
saveGoals([ /* ■■■ ■■■■■■■■ ■■■■■■■ Goal */ ]);
}
```

Модуль utils.js — вспомогательные функции

Утилиты используются несколькими модулями — вынесены в отдельный файл по принципу DRY (Don't Repeat Yourself).

```
function generateId() {   return
Date.now().toString() +
Math.random().toString(36).substr(2, 9); }

function formatDate(dateString) {
  return new Date(dateString).toLocaleDateString('ru-RU');
}

function calculateCompletedGoals(goals) {
return goals.filter(g => g.completed).length;
}

function getCategoryStats(goals) {
return goals.reduce((acc, g) => {
    acc[g.category] = (acc[g.category] || 0) +
1;     return acc;   }, {}); }

function getMostPopularCategory(stats) {
return Object.entries(stats)
.reduce((max, [cat, cnt]) =>
      cnt > max[1] ? [cat, cnt] : max, ['', 0])[0];
}
```

Модуль main.js — основная логика приложения main.js — самый объёмный модуль (2 863 байт). Загружается только на главной странице index.html.

Функция renderGoals()

Получает массив из getGoals() и генерирует HTML-карточки. Если массив пуст — показывает сообщение «Цели не найдены». Вызывается при загрузке страницы и после каждого изменения данных.

Функция createGoalCard(goal)

Создаёт DOM-элемент div.goal-card через innerHTML. Карточка содержит: заголовок, метку категории, иконку приоритета (/ /), дедлайн, прогресс-бар, кнопки «Выполнена» и «Удалить».

Функция deleteGoal(goalId)

Запрашивает подтверждение через window.confirm(). При согласии фильтрует массив, исключая цель по id, сохраняет и перерисовывает список.

Функция toggleGoalStatus(goalId)

Находит цель через Array.find(), инвертирует поле completed (true → false, false → true), вызывает saveGoals() и renderGoals().

Функция filterGoals()

Читает значения трёх фильтров (#search, #categoryFilter, #statusFilter) и применяет цепочку Array.filter() без мутации данных в localStorage. Вызывается на событие input/change каждого элемента управления.

```
function filterGoals() {
let goals = getGoals();
  const text = document.getElementById('search')
.value.toLowerCase();
  const cat  = document.getElementById('categoryFilter').value;
const stat = document.getElementById('statusFilter').value;

  if (text) goals = goals.filter(
    g => g.title.toLowerCase().includes(text));
if (cat && cat !== 'all')
    goals = goals.filter(g => g.category === cat);
if (stat === 'active')
    goals = goals.filter(g => !g.completed);
if (stat === 'completed')
    goals = goals.filter(g => g.completed);

  renderGoalsList(goals); }
```

Модуль form.js — обработка формы

Управляет созданием цели: перехватывает submit, валидирует поля, формирует объект Goal и сохраняет его в localStorage.

```
function validateForm() {
  const title    = document.getElementById('title').value.trim();
const category = document.getElementById('category').value;   const
deadline = document.getElementById('deadline').value;   const
priority = document.getElementById('priority').value;   const
progress = parseInt(
    document.getElementById('progress').value);
  if (!title)    { alert('[redacted]'); return false; }
if (!category) { alert('[redacted]'); return false;
}   if (!deadline) { alert('[redacted]'); return false;
}   if (!priority) { alert('[redacted]'); return
false; }   if (isNaN(progress)||progress<0||progress>100)
                 { alert('[redacted]: 0-100'); return false; }
return true;
}
```

Интеграция хранилища данных и обработка событий

Поток данных в приложении

Взаимодействие модулей построено на принципе однонаправленного потока данных. Пользователь взаимодействует с UI → JS-обработчик вызывает функцию слоя данных → данные обновляются в localStorage → UI перерисовывается.

```
[redacted] → [HTML-[redacted]] → form.js
  ↓ validateForm() + createGoalObject()
storage.js → saveGoals() → localStorage["goals"]
  ↓ getGoals()
main.js → renderGoals() → createGoalCard() → DOM
  ↓ [redacted] [redacted] deleteGoal() /
toggleGoalStatus() → saveGoals()
  ↓ stats.js → renderStats()
calculateCompletedGoals()
getCategoryStats()
```

Регистрация обработчиков событий

Все обработчики регистрируются в событии DOMContentLoaded, гарантируя полную загрузку DOM перед выполнением скриптов.

```
document.addEventListener('DOMContentLoaded', () => {
initializeGoals(); // [redacted] [redacted] [redacted]
renderGoals();     // [redacted] [redacted]

  // [redacted]
document.getElementById('search')
.addEventListener('input', filterGoals);
document.getElementById('categoryFilter')
.addEventListener('change', filterGoals);
document.getElementById('statusFilter')
.addEventListener('change', filterGoals); });
```

Делегирование событий для кнопок карточек

Кнопки в карточках создаются динамически при каждом renderGoals(). Обработчики привязываются напрямую к кнопкам через атрибуты data-id и onclick (inline), что исключает проблему потери слушателей при перерисовке списка.

```
// [redacted] [redacted] [redacted] (createGoalCard):
`
```

```
  [redacted]



  ${goal.completed ? '[redacted]' : '[redacted]'}
 `
```

Модуль stats.js — интеграция аналитики stats.js загружается только на stats.html. После вызова getGoals() применяет утилиты из utils.js для подсчёта метрик и формирует HTML-блок карточек статистики.

```
function renderStats() {   const
goals     = getGoals();   const
total     = goals.length;
  const completed = calculateCompletedGoals(goals);
const active    = total - completed;   const pct =
total > 0     ? Math.round((completed / total) *
100) : 0;   const catStats =
getCategoryStats(goals);   const topCat   =
getMostPopularCategory(catStats);

  document.getElementById('statsContainer').innerHTML = `


      ■■■■■ ■■■■■${total}



      ■■■■■■■■■■${completed} (${pct}%)


    ...`;
}
```

Тестирование и отладка приложения

Виды тестирования

В ходе разработки применялось ручное функциональное тестирование. Каждая функция проверялась после реализации; критические сценарии проверялись повторно после изменений в коде.

Тест кейсы – Результаты функционального тестирования всех ключевых сценариев работы приложения представлены в Таблице **8.**

№	Сценарий	Ожидаемый результат	Статус
1	***1***	***2***	***3***
ТК-1	Создание цели с корректными данными	Цель сохраняется, редирект на главную	Пройден
ТК-2	Попытка сохранить цель без названия	Появляется alert с ошибкой	Пройден
ТК-3	Прогресс = 150	alert «Прогресс: 0-100»	Пройден
ТК-4	Поиск по тексту в поле «Бег»	Отображаются только цели с «Бег» в названии	Пройден
ТК-5	Фильтр: категория Sport	Видны только цели категории Sport	Пройден
ТК-6	Переключение статуса цели	Статус инвертируется, карточка обновляется	Пройден
ТК-7	Удаление цели с confirm «Отмена»	Цель остаётся в списке	Пройден
ТК-8	Страница статистики без целей	Все метрики = 0, ошибки деления нет	Пройден
ТК-9	Адаптив: ширина окна 375px	Карточки в 1 колонку, меню переносится	Пройден
ТК-10	Обновление страницы	Данные сохранены из localStorage	Пройден
Примечание: Все 10 тест-кейсов пройдены успешно. Статус «Пройден» означает полное соответствие фактического результата ожидаемому.			

Обнаруженные и исправленные ошибки

– Ошибка: getGoals() возвращала null при первом запуске → Исправление: добавлена проверка «data ? JSON.parse(data) : []»;
– Ошибка: деление на ноль в расчёте процента при отсутствии целей → Исправление: условие «total > 0 ? ... : 0»;
– Ошибка: фильтр применялся с задержкой → Исправление: слушатель input вместо change для поля поиска.

Инструменты отладки

– Chrome DevTools → Console: логирование состояния массивов;
– Chrome DevTools → Application → localStorage: проверка содержимого хранилища;
– Chrome DevTools → Responsive Mode: тестирование адаптивности.

ЗАКЛЮЧЕНИЕ

В ходе учебной практики по модулю ПМ2 «Прикладное программирование» разработано и протестировано веб-приложение GoalTracker — система постановки и отслеживания личных целей.

Реализованный функционал

- создание целей с полным набором атрибутов через HTML-форму свалидацией;
- отображение целей в адаптивной сетке карточек;
- поиск по названию в реальном времени;
- фильтрация по категории и статусу;
- переключение статуса цели (активна / выполнена);
- удаление цели с подтверждением;
- статистика с ключевыми метриками;
- сохранение данных в localStorage между сессиями;– адаптивный интерфейс (mobile-first).

Приобретённые компетенции

- HTML5: семантическая разметка, формы, data-атрибуты;
- CSS3: Grid Layout, Flexbox, анимации, адаптивный дизайн;
- JavaScript ES6+: модульность, работа с массивами, DOM API,события;
- Web Storage API: localStorage, JSON-сериализация;
- Git / GitHub: контроль версий, удалённый репозиторий;
- Архитектура: SoC, DRY, событийно-ориентированное программирование.

Итоги практики

Все задачи практики выполнены в полном объёме. Разработан функциональный программный продукт, освоены ключевые технологии клиентской веб-разработки. Полученные навыки составят основу для дальнейшей профессиональной деятельности в области разработки программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. MDN Web Docs. HTML: HyperText Markup Language [Электронныйресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/HTML – Дата обращения: 10.04.2026.
2. MDN Web Docs. CSS: Cascading Style Sheets [Электронный ресурс]. –Режим доступа: https://developer.mozilla.org/ru/docs/Web/CSS – Дата обращения: 10.04.2026.
3. MDN Web Docs. JavaScript [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/JavaScript – Дата обращения: 10.04.2026.
4. MDN Web Docs. Web Storage API [Электронный ресурс]. – Режимдоступа: https://developer.mozilla.org/ru/docs/Web/API/Web_Storage_API – Дата обращения: 10.04.2026.
5. MDN Web Docs. DOM (Document Object Model) [Электронныйресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/API/Document_Object_Model – Дата обращения: 09.04.2026.
6. W3Schools. JavaScript Tutorial [Электронный ресурс]. – Режимдоступа: https://www.w3schools.com/js/ – Дата обращения: 08.04.2026.
7. Флэнаган Д. JavaScript: Подробное руководство. 7-е изд. – СПб.: Питер, 2021. – 1100 с.
8. Хавербеке М. Выразительный JavaScript. 3-е изд. – М.: Диалектика,2022. – 480 с.
9. CSS-Tricks. A Complete Guide to CSS Grid [Электронный ресурс]. – Режим доступа: https://css-tricks.com/snippets/css/complete-guide-grid/ – Дата обращения: 07.04.2026.
10. CSS-Tricks. A Complete Guide to Flexbox [Электронный ресурс]. – Режим доступа: https://css-tricks.com/snippets/css/a-guide-to-flexbox/ – Дата обращения: 07.04.2026.
11. Google Fonts. Inter Font Family [Электронный ресурс]. – Режимдоступа: https://fonts.google.com/specimen/Inter – Дата обращения: 05.04.2026.
12. GitHub Docs. About Git [Электронный ресурс]. – Режим доступа: https://docs.github.com/ru/get-started/using-git/about-git – Дата обращения: 10.04.2026.

ПРИЛОЖЕНИЯ

Приложение А – Листинг storage.js

```
const STORAGE_KEY = 'goals';

function getGoals() {
  const data =
localStorage.getItem(STORAGE_KEY);   return data
? JSON.parse(data) : []; }

function saveGoals(goals) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(goals));
}

function initializeGoals() {
  if (localStorage.getItem(STORAGE_KEY))
return;   const defaultGoals = [     {
      id: generateId(),
      title: '[redacted] JavaScript',
category: 'Study', deadline: '2026-05-01',
priority: 'High', progress: 60,
      description: '[redacted]
GoalTracker',       completed: false, createdAt:
'2026-01-15'     },     {
      id: generateId(),       title: '[redacted]
3 [redacted]',       category:
'Health', deadline: '2026-04-30',       priority:
'Medium', progress: 30,       description: '',
      completed: false, createdAt: '2026-01-
20'     },     {
      id: generateId(),       title: '[redacted]
[redacted]',       category:
'Finance', deadline: '2026-07-01',
priority: 'Low', progress: 50,       description:
'[redacted] 80 000 [redacted]',       completed: false,
createdAt: '2026-02-01'     }   ];
  saveGoals(defaultGoals);
}
```

Приложение Б – Листинг utils.js

```
function generateId() {   return Date.now().toString() +
Math.random().toString(36).substr(2, 9); }

function formatDate(dateString) {
```

```
  const date = new Date(dateString);
return date.toLocaleDateString('ru-RU');
}

function calculateCompletedGoals(goals) {
return goals.filter(g => g.completed).length;
}

function getCategoryStats(goals) {
return goals.reduce((acc, goal) => {
    acc[goal.category] = (acc[goal.category] || 0) +
1;     return acc;   }, {}); }

function getMostPopularCategory(stats) {   return
Object.entries(stats).reduce(     (max, [cat, cnt]) =>
cnt > max[1] ? [cat, cnt] : max,
    ['', 0]
  )[0];
}
```

Приложение В – Структура данных (пример JSON)

```
[
  {
    "id":          "1712345678abcdef",
    "title":       "[redacted] [redacted] [redacted] JavaScript",
    "category":    "Study",
    "deadline":    "2026-05-01",
    "priority":    "High",
    "progress":    60,
    "description": "[redacted] [redacted] GoalTracker",
    "completed":   false,
"createdAt":   "2026-01-15"
  }
]
```